

$(N, 3) \rightarrow (N, 3, 10) \rightarrow (N, 30) \rightarrow (N, 200) \rightarrow (N, 27)$

NN ZERO TO HERO • MAKEMORE PART 2

从 Bigram 到 MLP

字符级语言模型的实现、调参与 embedding 可视化

复现 Bengio et al. 2003 • A Neural Probabilistic Language Model | leiwingqueen • 2026.09

问题

Bigram 的天花板：想多看几个字符，表格就爆炸

19,683

看 3 个字符的计数表要 27^3 行，绝大多数格子计数为 0，无法泛化

11,897

MLP 的全部参数 (block 3 · emb 10 · hidden 200)，相似字符共享统计强度

思路：每个字符 → 低维稠密向量 (embedding)，上下文向量拼接后交给 MLP 预测下一个字符。part1 的 bigram dev loss 是 2.4544，这是本次要打败的基线。

五步前向：查表、拼接、隐藏层、输出



C	W1	b1	W2	b2
$27 \times 10 = 270$	$30 \times 200 = 6,000$	200	$200 \times 27 = 5,400$	27

滑动窗口：一个名字拆成 len + 1 条样本

以 "emma"、block_size = 3 为例

上下文	X	Y
...	[0, 0, 0]	e (5)
..e	[0, 0, 5]	m (13)
.em	[0, 5, 13]	m (13)
emm	[5, 13, 13]	a (1)
mma	[13, 13, 1]	. (0)

最后一行预测结束符 '!', 模型靠它学会 “何时停下”

train 80%

算梯度、更新参数

dev 10%

调超参，判断欠拟合 / 过拟合

test 10%

只在最后看一次，避免被调参污染

前向传播

四行代码完成前向

```
def forward(X, params):
    N = X.shape[0]
    C, W1, b1, W2, b2 = params
    emb = C[X]                      # (N, 3, 10)
    hidden = torch.tanh(
        emb.view(N, -1) @ W1 + b1) # (N, 200)
    logits = hidden @ W2 + b2      # (N, 27)
    return logits
```

C[X] 高级索引

等价于 one-hot @ C，但查表快得多

view 而不是 cat

只换 shape / stride，不复制内存，O(1)

N 不写死

训练 32 行、评估 18 万行共用同一个函数

训练循环

minibatch + 学习率衰减

```
for i in range(steps):
    ix = torch.randint(0, N, (batch_size, ), generator=g)
    logit = forward(Xtr[ix], params)
    loss = F.cross_entropy(logit, Ytr[ix])
    for p in params:
        p.grad = None
    loss.backward()
    lr_i = lr if i < decay else lr * decay_factor
    for p in params:
        p.data += -lr_i * p.grad
```

minibatch

梯度不准但便宜，多走步数更划算

F.cross_entropy

先减每行最大值再 exp，大 logit 不会溢出成 nan

grad 置 None

PyTorch 梯度会累加，每步都要清

lr decay

前 60% 步用 0.1，之后 $\times 0.1$ 精调

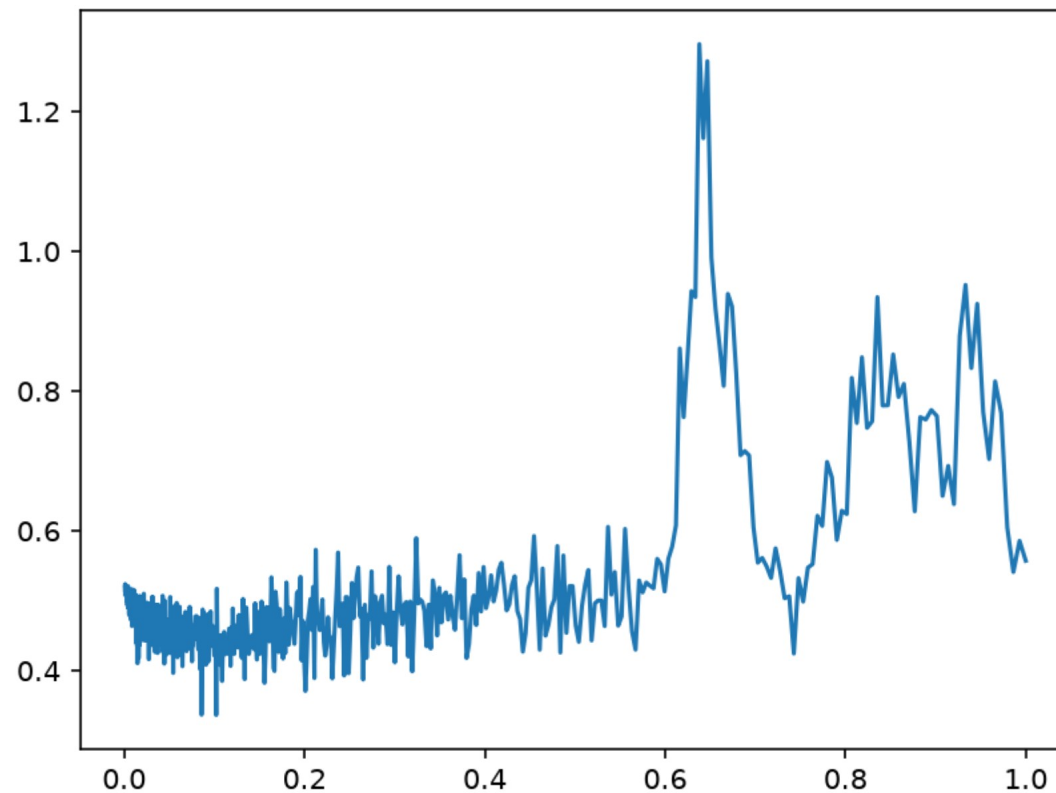
学习率搜索

在指数上均匀扫描，找震荡开始前的 lr

```
lre = torch.linspace(-3, 0, 1000)
lrs = 10 ** lre
# 第 i 步用 lrs[i] 更新一次
```

学习率的影响是乘性的：0.001→0.01 和 0.1→1 是同一量级的差别，所以在指数上取值。

每次扫描都用 `init_params()` 重新初始化一套参数，不污染正式训练。结论与课程一致：**lr = 0.1** 是好的起点。



实测 (E04 配置)：横轴 lr，纵轴 $\log_{10}(\text{loss})$ ；lr 超过约 0.6 后 loss 飙升

基线结果

10 万步后，MLP 把 dev loss 压低了 0.25

part1 bigram · dev

2.4544

MLP · dev



2.2054

MLP · train

2.1716

train 和 dev 只差 0.034 → 仍是欠拟合，模型还有加大的空间。这就是 E01 调参的出发点。

练习 E01

调参目标：dev loss < 2.17

可动的旋钮：训练步数 · N_EMB · N_HIDDEN · BLOCK_SIZE

规则：每次只改一个，只看 dev，test 留到最后

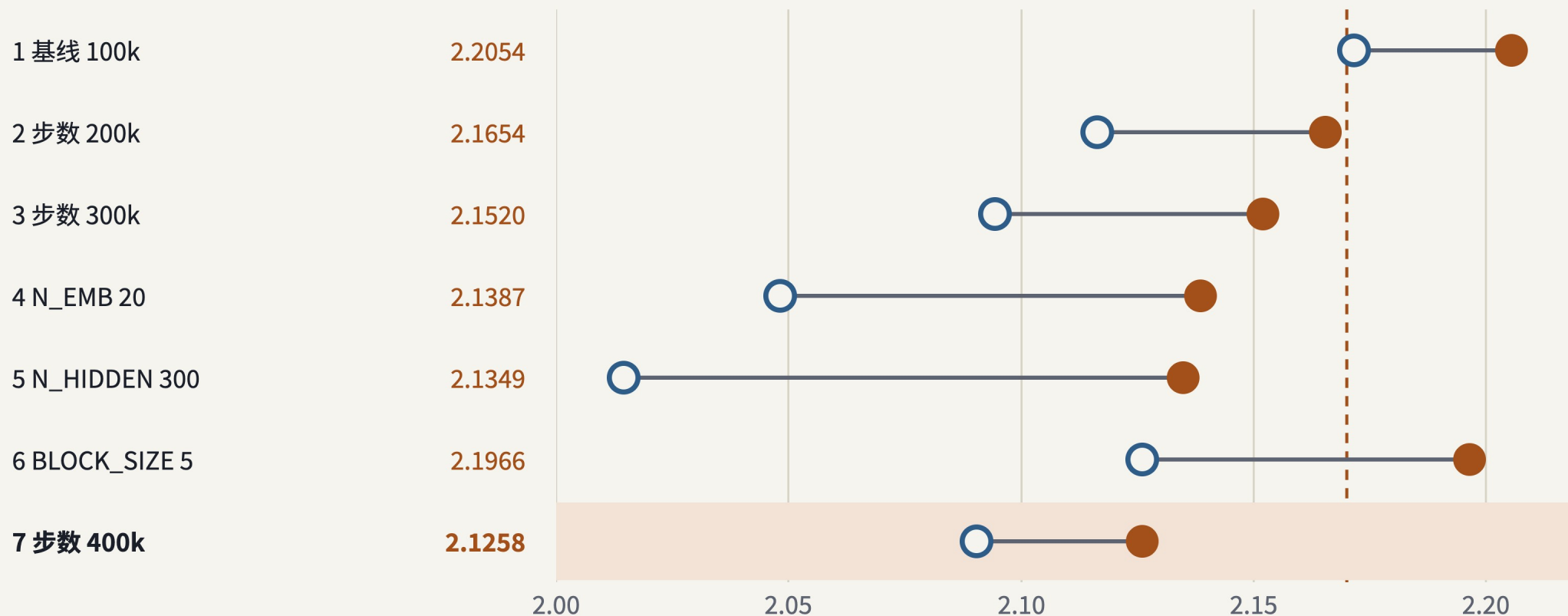
七组实验，每次只改一个旋钮

#	改动	train	dev	gap	观察
1	基线 100k 步	2.1716	2.2054	0.034	欠拟合，起点
2	步数 → 200k	2.1163	2.1654	0.049	两者都降，仍在收敛
3	步数 → 300k	2.0944	2.1520	0.058	dev 只降 0.013，加步数收益变小
4	N_EMB 10 → 20	2.0482	2.1387	0.091	dev 更低，但 gap 明显拉大
5	N_HIDDEN 200 → 300	2.0144	2.1349	0.121	train 最低，过拟合信号最强
6	BLOCK_SIZE 3 → 5	2.1260	2.1966	0.071	train 反升，gap 收窄
7	步数 → 400k（最终）	2.0905	2.1258	0.035	dev 最低，gap 回到基线水平

E01 · 可视化

加容量压低了 train，更长的上下文才压低 dev

○ 空心 = train loss ● 实心 = dev loss 连线长度 = gap 虚线 = 目标 2.17



E01 顺手做了 E02

缩放初始化：初始 loss 回到“随机猜”的水平

```
W1 = randn(fan_in, n_hidden) * (5/3) / fan_in**0.5
b1 = randn(n_hidden) * 0.01
W2 = randn(n_hidden, 27) * 0.01
b2 = randn(27) * 0
```

W2 × 0.01、b2 置 0：logits 初始都接近 0，softmax 接近均匀分布，不再“自信地猜错”。

W1 × (5/3)/√fan_in：tanh 的 Kaiming 增益，避免隐藏层一开始就饱和。

首步 loss

3.2957

理论值 $-\ln(1/27) = 3.2958$
未缩放时课程中是 20+

复盘

一个隐藏变量：batch_size 传成了 BLOCK_SIZE

```
BATCH_SIZ = 64 # 定义了，但没被用到
```

```
lossi = train(Xtr, Ytr, params,  
              steps=TRAIN_STEP,  
              batch_size=BLOCK_SIZE,  
              lr=LEARNING_RATE)
```

影响

当前 e01 / e04 脚本每步只用 3 或 5 条样本，而不是 32 或 64

混淆

“BLOCK_SIZE 3→5 让 train 变高” 其实同时改了 batch 3→5

下一步

改为 batch_size=BATCH_SIZ，重跑第 5、6、7 组做对照

练习 E04

网络自己学会了“元音是一类”

把 N_EMB 设为 2，训练后直接把 C 的 27 行画在平面上：

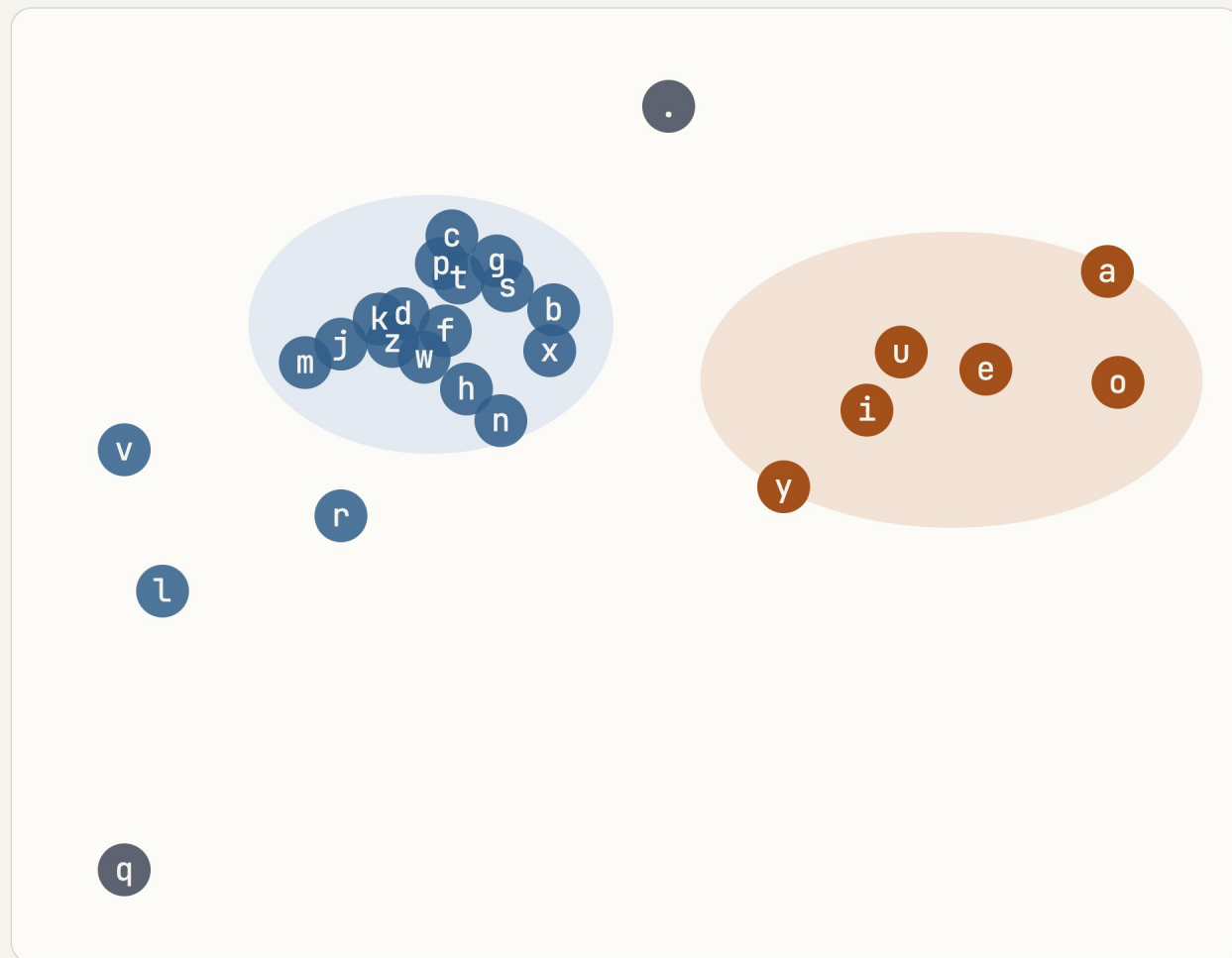
| **a e i o u y** 聚在右侧

| 常见辅音挤在中间一团

| **q** 被甩到角落，**!** 自成一派

block 5 • hidden 300 • 100k 步

train 2.3199 • dev 2.3152 (2 维太窄，只作可视化)



总结

dev loss: 2.4544 → 2.2054 → 2.1258

实现

embedding 查表 + view 拼接 + MLP,
用不到 1.2 万参数打败 bigram

调参

盯住 train/dev gap: 加容量易过拟合,
加上下文配合更多步数才降 dev

可视化

2 维 embedding 里元音自动聚类, 相似性是学出来的

下一步

① 修正 batch_size 后重跑对照 ② E03: 试试 Bengio 论文中输入直连输出的 skip connection ③ part3: 激活值、梯度与 BatchNorm